PPAD (PPGCC) - Otimizando o desempenho de códigos para afinidade de memória, vetorização e eficiência energética

Integrantes: até dois alunos
Entrega: 26/10
Instruções: *O desempenho e o consumo energético podem ser medidos em equipamentos institucionais da Unifesp ou no próprio equipamento pessoal ou profissional. Deve-se especificar o equipamento utilizado (modelo de CPU, quantidade de memória interna, suporte a conjuntos de instruções vetoriais, como AVX2/AVX-512, e a ferramenta/interface utilizada para medição de energia, ex.: perf com eventos RAPL, likwid-powermeter, Intel Power Gadget, Powercap ou ferramenta equivalente).*
A atividade pode ser realizada de duas formas/opções possíveis:

Opção 1: Trazer código-fonte de interesse (Afinidade de Cache + Vetorização + Eficiência Energética)

Isolar e preparar um trecho (considerado pequeno) de um código-fonte de interesse (para projeto de pesquisa da pós-graduação) em C/C++ ou Fortran, com seu respectivo conjunto de dados de entrada, cuja execução gaste, pelo menos, na casa de dezena de segundos. O código deve possuir laços intensivos passíveis de vetorização e de otimização de localidade de memória.
A atividade consiste em aplicar e avaliar o impacto de forma **isolada/separada** para:

1. Otimizações de **afinidade/localidade de memória** (reorganização de laços se possível, alinhamento de dados e/ou blocagem/tiling).
2. Otimizações de **vetorização SIMD** (utilizando pragmas em C/C++ como `#pragma omp simd` ou `#pragma vector`, ou flags explícitas do compilador).
3. A combinação de ambas as técnicas.

Fazer medidas de desempenho e consumo energético relacionadas a: tempo total de processamento, quantidade de *Cache-Misses* em memória Cache L1, L2, L3, MFLOPS (se possível), CPI (*Cycles per Instruction*), **potência média consumida (Watts)**, **energia total consumida (Joules)** e a métrica de eficiência energética equivalente adequada ao problema (ex.: **MFLOPS/Watt**, **MFLOPS/Joules**, ou **Pixels Processados por Joule/Watt**).

A entrega da Opção 1 consiste em um **arquivo compactado** contendo:

1. Dois ou mais **arquivos relacionados ao código-fonte utilizado** (versão base, versão com afinidade de cache, versão vetorizada com pragmas, versão combinada), o **conjunto de dados de entrada** e,
2. Um **relatório** que deve conter uma breve descrição sobre o código-fonte utilizado, explicando para qual finalidade ele foi criado, detalhando os pragmas, ferramentas de

medição de energia e alterações utilizadas, apresentando a análise comparativa dos dados de desempenho e eficiência energética.

Opção 2: Atividade prática de programação (Filtro de Convolução 2D / Stencil com Eficiência Energética)

Nesta opção, o problema estudado será a aplicação de um **filtro de convolução 2D (Stencil)** sobre matrizes/imagens de entrada. Os alunos devem avaliar e medir o impacto da **afinidade de memória/cache**, da **vetorização** e do **consumo/eficiência energética** de forma estritamente **separada**, permitindo isolar a contribuição de cada técnica tanto no desempenho computacional quanto na eficiência energética do sistema.

Exercícios da Opção 2:

1. **Afinidade de Memória (Reorganização de Laços - Sem Vetorização):**
   Implemente, em linguagem C/C++, o algoritmo baseline de convolução 2D usando uma máscara 3x3 (ex.: Blur Gaussiano ou Sobel). Altere a hierarquia/ordem dos laços de varredura para comparar o acesso à memória:
   - Varredura por linhas (*Row-Major* - acesso contíguo).
   - Varredura por colunas (*Column-Major* - acesso não contíguo).

   Desabilite a vetorização do compilador (ex.: utilizando a flag -fno-tree-vectorize no GCC/Clang).
   Faça medições de desempenho e consumo de energia variando o tamanho **N*N** da matriz/imagem em **1024x1024, 2048x2048 e 4096x4096**.
   Crie tabelas e gráficos indicando: tempo total de processamento, quantidade de *Cache-Misses* em memória Cache L1, L2, L3, MFLOPS, CPI, **potência (Watts)**, **energia total (Joules)** e a eficiência energética em **MFLOPS/Watt** (ou **MFLOPS/Joule**).
   Destaque a forma de organização do laço mais eficiente em termos de localidade espacial, afinidade de cache e consumo de energia.

2. **Vetorização Isolada (Uso de Pragmas em C - Sem Blocagem):**
   Mantendo a melhor organização de laços obtida no exercício 1, aplique técnicas de **vetorização SIMD** utilizando pragmas na linguagem C (como `#pragma omp simd` ou `#pragma vector`) e/ou alinhamento de memória (`aligned_alloc` / `__attribute__((aligned(32)))`).
   Habilite as flags de vetorização do compilador (ex.: `-O3 -mavx2` ou `-fopenmp-simd`).
   Atenção: Esta etapa deve ser realizada **sem alterar a estrutura de blocagem de memória**, para medir exclusivamente o ganho trazido pelas instruções vetoriais (SIMD).
   Refaça os experimentos para os mesmos tamanhos N de matriz do exercício 1 e compare diretamente os MFLOPS, CPI, tempo e a métrica de eficiência energética (**MFLOPS/Watt**) com os resultados não-vetorizados.

3. **Blocagem de Memória (*Tiling*) Isolada e Combinada:**
   a. Aplique o algoritmo com blocagem (*tiling*) na versão sequencial (sem pragmas de

vetorização), definindo o tamanho do bloco para **8, 16, 32 e 64**, variando-se o tamanho N da matriz da mesma maneira que no exercício 1. Identifique o tamanho do bloco mais eficiente em termos de desempenho e uso de energia para a hierarquia de cache do seu processador.

b. Em seguida, aplique os pragmas de vetorização (do exercício 2) **juntamente** com a técnica de blocagem (versão combinada). Compare o consumo em Watts e a eficiência em MFLOPS/Watt obtidos em relação às versões anteriores.

Importante: Demonstre no relatório que o código está livre de erros no acesso aos dados nos limites das bordas para qualquer tamanho do bloco e da imagem.

4. **Comparação com Biblioteca Otimizada (Desempenho x Eficiência Energética):** Utilize uma função equivalente de convolução/filtro 2D de uma biblioteca otimizada (como **OpenCV**, **Intel IPP** ou rotinas de stencil/BLAS aplicáveis) com as otimizações automáticas habilitadas na biblioteca e refaça os experimentos para os mesmos tamanhos de matriz.
   Houve melhora significativa em relação ao desempenho e à eficiência energética (MFLOPS/Watt) obtidos na sua versão otimizada (blocada + vetorizada)? Analise o nível de utilização dos recursos de hardware e o comportamento do consumo energético em Watts.

A entrega da Opção 2 consiste em um **arquivo compactado** contendo:

1. **Arquivos com os códigos-fonte utilizados** (baseline, reorganizado, vetorizado com pragmas, blocado e biblioteca) e,
2. **Um relatório** completo com os dados e análise comparativa das medidas de desempenho e eficiência energética solicitadas, isolando o impacto da afinidade de cache, da vetorização e o impacto no consumo em Watts e Joules.

Dica de Implementação para a opção 2, item 4 (Uso do OpenCV em C++):

Para facilitar a realização do item 4, pode-se utilizar a biblioteca **OpenCV** (cv::filter2D) em C++, que já possui vetorização e paralelismo de hardware altamente otimizados (pode ser necessário antes instalar pacotes para usar o OpenCV). Exemplo de estrutura do código:

```
#include <iostream>
#include <chrono>
#include <opencv2/opencv.hpp>

int main() {
    int largura = 4096, altura = 4096;

    // 1. Alocação de matrizes em ponto flutuante (CV_32F = float)
    cv::Mat imagem_in(altura, largura, CV_32F);
    cv::Mat imagem_out(altura, largura, CV_32F);
    cv::randu(imagem_in, cv::Scalar(0.0f), cv::Scalar(255.0f));
```

```
    // 2. Definir a máscara de convolução 3x3 (Blur Gaussiano)
    cv::Mat mascara = (cv::Mat_<float>(3, 3) <<
        1.0f/16, 2.0f/16, 1.0f/16,
        2.0f/16, 4.0f/16, 2.0f/16,
        1.0f/16, 2.0f/16, 1.0f/16
    );

    // 3. Processamento com a biblioteca otimizada
    auto inicio = std::chrono::high_resolution_clock::now();
    cv::filter2D(imagem_in, imagem_out, -1, mascara);
    auto fim = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double> tempo = fim - inicio;
    std::cout << "Tempo OpenCV: " << tempo.count() << " segundos.\n";

    return 0;
}
```

Compilação no Linux:
g++ -O3 script_opencv.cpp -o script_opencv `pkg-config --cflags --libs opencv4`